Trabajo Práctico

Bases de Datos I — Segundo Semestre 2023

1 Modelo de Datos

A continuación se presenta el modelo de datos que se utiliza para almacenar información relativa a pedidos de clientes al supermercado *ChangoOnline*.

```
cliente(
    id_usuarie:int,
    nombre:text,
    apellido:text,
    dni:int;
    fecha_nacimento:date,
    telefono:char(12),
    email:text --válido
)

direccion_entrega(
    id_usuarie:int,
    id_direccion_entrega:int,
    direccion:varchar(60), --incluye calle, nro, piso, y dpto
    localidad:varchar(30),
    codigo_postal:char(8) --necesario para obtener la tarifa de entrega
)

tarifa_entrega(
    codigo_postal_corto:char(4), --dígitos 2 a 5 del cp completo
    costo:decimal(12,2) --el costo de entrega por cp corto
)

producto(
    id_producto:int,
    nombre:text,
    precio_unitario:decimal(12,2),
    stock_disponible:int, --no incluye cantidad reservada para pedidos
    stock_reservado:int, --cantidad incluida en pedidos
    punto_reposicion:int,
                --reponer si disponible + reservado <= punto_reposicion
    stock_maximo:int --hasta cuánto reponer
)
```

```
pedido(
    id_pedido:int,
    f_pedido:timestamp,
    fecha_entrega:date,
    hora_entrega_desde:time,
    hora_entrega_hasta:time,
    id_usuarie:int,
    id_direccion_entrega:int,
    monto_total:decimal(12,2), --actualizado de acuerdo al detalle
    costo_envio:decimal(12,2), --de acuerdo a la dirección de entrega
    estado:char(10) --`ingresado',`completado',`cancelado',`entregado'
)

pedido_detalle(
    id_pedido:int,
    id_producto:int,
    cantidad:int,
    precio_unitario:decimal(12,2)
)

reposicion(
    id_producto:int,
    fecha_solicitud:date,
    cantidad_a_reponer:int,
    fecha_reposicion:date, --se graba al efectuarse la reposición
    estado:char(12) --`pendiente', `finalizada'
)

error(
    id_error:int,
    id_pedido:int,
    f_pedido:timestamp,
    id_usuarie:int,
    id_direccion_entrega:int,
    direccion:varchar(60),
    localidad:varchar(30),
    codigo_postal:char(8),
    id_producto:int,
    cantidad:int,
    operacion:char(12),
             --`creación', `producto', `cierre', `cancelación', `entrega'
    f_error:timestamp,
    motivo:varchar(64)
)
```

```
envio_email(
    id_email:int,
    f_generacion:timestamp,
    email_cliente:text,
    asunto:text,
    cuerpo:text,
    f_envio:timestamp,
    estado:char(10) --`pendiente', `enviado'
)

-- Esta tabla *no* es parte del modelo de datos, pero se incluye para
-- poder probar las funciones.
entrada_trx_pedido(
    id_orden:int, --en qué orden se ejecutarán las transacciones
    operacion:char(12),
             --`creación', `producto', `cierre', `cancelación', `entrega'
    id_usuarie:int,
    id_direccion_entrega:int,
    id_pedido:int,
    id_producto:int,
    cantidad:int,
    fecha_hora_entrega:timestamp
)
```

El sistema debe administrar el ingreso, la cancelación, y la entrega de pedidos, mantener toda la información de les clientes y de los productos, y controlar que el stock disponible y el reservado de los productos sea actualizado correctamente—e.g. no puede agregarse a los pedidos una cantidad mayor a la disponible, el proceso de entrega del pedido no puede quedar a medio hacer, etc.

Les clientes deben ser informados vía email cuando ocurran eventos importantes sobre sus pedidos. Además, es necesario poder generar solicitudes de reposición para aquellos productos que hayan quedado con bajo stock.

2 Creación de la Base de Datos

La base base de datos deberá nombrarse con el apellido de cada integrante, separados con underscores, seguidos del string `_db1`, e.g. `giunta_maradona_palermo_riquelme_db1`.

Se deberán crear las tablas respetando ***exactamente*** *los nombres de tablas, atributos, y tipos de datos especificados.*

Se deberán agregar las PK's y FK's de todas las tablas, por separado de la creación de las mismas. Además, se deberá tener la posibilidad de borrar todas las PK's y FK's.

3 Instancia de los Datos

Se deberán cargar 20 clientes y 20 productos. Todes les clientes tendrán una dirección de entrega, excepto dos clientes que tendrán dos direcciones cada une. La tabla `tarifa_entrega` deberá tener los códigos postales de todes les clientes, excepto para tres de las direcciones.

> **Observación:**
>
> Los datos de les clientes, de sus direcciones de entrega, de los productos, de las tarifas de entrega, así como los datos de entrada para las transacciones de prueba, deberán cargarse en las tablas correspondientes a partir de los siguientes documentos JSON:
>
> - `clientes.json`
> - `direcciones.json`
> - `productos.json`
> - `tarifas_entrega.json`
> - `trx_pedidos.json`
>
> Estos archivos `json` se encuentran en el directorio compartido de Google Drive de la comisión, junto a este archivo `pdf`.

4 Stored Procedures y Triggers

El trabajo práctico deberá incluir los siguientes stored procedures ó triggers:

- **creación del pedido:** se deberá incluir la lógica que reciba los datos del pedido—id de le usuarie, id de dirección—y que devuelva `true` si se logra crear el pedido, ó `false` si se rechaza. El procedimiento deberá validar los siguientes elementos antes de confirmar el alta:
  - Que el id de le usuarie exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de usuarie no válido`.
  - Que el id de la dirección de entrega exista para le cliente. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de dirección no válido`.
  - Que el código postal de la dirección de entrega tenga una tarifa establecida. En caso de que no cumpla, se debe cargar un error con el mensaje `?dirección de entrega fuera del área de atención`.

  Si se aprueba la creación del pedido, se deberá insertar una fila en la tabla `pedido` con los datos de le cliente, la fecha y hora del pedido, y el costo de envío correspondiente al código postal, dejando su estado como `ingresado`.
- **agregado de producto:** se deberá incluir la lógica que reciba un id de pedido, un id de producto y una cantidad, y añada la información al pedido. Retornará `true` si la

operación se realiza satisfactoriamente, ó `false` en caso contrario. El procedimiento deberá validar los siguientes elementos, antes de incorporar el ítem al pedido:

- Que el id de pedido exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de pedido no válido`.
- Que el pedido se encuentre en estado ingresado. En caso de que no cumpla, se debe cargar un error con el mensaje `?pedido cerrado`.
- Que el id de producto exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de producto no válido`.
- Que el producto tenga stock disponible para satisfacer la cantidad solicitada al momento de realizar el pedido. En caso de que no se cumpla, se debe cargar un nuevo error con el mensaje `?stock no disponible para el producto [id_producto]`, reemplazando en el mensaje `[id_producto]` por el valor correspondiente que generó el error.

Si se aprueba la incorporación, se deberá insertar una fila en la tabla `pedido_detalle` con los datos del producto. Si el pedido ya contiene al producto, se deberá actualizar la fila correspondiente, sumando la nueva cantidad. En cualquiera de los dos casos, se deberá descontar dicha cantidad del stock disponible y sumarla al stock reservado. También deberá mantenerse actualizado el monto total del pedido de forma coherente con el detalle de productos solicitados.

- **cierre del pedido:** se deberá proveer la lógica que reciba un id de pedido, la fecha y hora de entrega, y que devuelva `true` si se logra cerrar el pedido ó `false` en caso contrario. El procedimiento deberá validar los siguientes elementos antes de confirmar el cierre:
  - Que el id de pedido exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de pedido no válido`.
  - Que el pedido tenga al menos un producto agregado. En caso de que no cumpla, se debe cargar un error con el mensaje `?pedido vacío`.
  - Que la fecha y hora de entrega sea posterior a la fecha y hora actual. En caso de que no cumpla, se debe cargar un error con el mensaje `?fecha de entrega no válida`.

  Si se aprueba el cierre, se deberá actualizar la fila correspondiente en la tabla `pedido` con la fecha de entrega, y las horas de entrega desde y hasta, dejando su estado como completado. Calcular la 'hora hasta' como dos horas posterior a la 'hora desde'.
- **cancelación de pedido:** se deberá proveer la lógica que permita cancelar un pedido. El procedimiento debe recibir el id de pedido, y retornar `true` si se logra cancelar el pedido ó `false` en caso contrario. El procedimiento deberá validar los siguientes elementos antes de confirmar la cancelación:

- Que el id de pedido exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de pedido no válido`.
- Que el pedido se encuentre en estado ingresado o completado. En caso de que no cumpla, se debe cargar un error con el mensaje `?pedido ya entregado o cancelado`.

Si las validaciones pasan correctamente, se deberá marcar el estado del pedido como cancelado. Además, deberá sumarse al stock disponible de cada producto la cantidad incluida en el pedido cancelado, y descontarla del stock reservado.

- **entrega de pedido:** se deberá proveer la lógica que reciba un id de pedido, y que retorne `true` si se logra marcar el pedido como entregado ó `false` en caso contrario. El procedimiento deberá validar los siguientes elementos antes de confirmar la entrega:
  - Que el id de pedido exista. En caso de que no cumpla, se debe cargar un error con el mensaje `?id de pedido no válido`.
  - Que el pedido se encuentre en estado completado. En caso de que no cumpla, se debe cargar un error con el mensaje `?pedido sin cerrar o ya entregado`.

  Si las validaciones pasan correctamente, se deberá marcar el estado del pedido como entregado. Además, deberá restarse del stock reservado de cada producto la cantidad incluida en el pedido entregado.
- **generación de solicitudes de reposición:** Diariamente se deberán generar las solicitudes de reposición de los productos que tengan un stock bajo. Para ello, se debe insertar una fila en la tabla `reposicion` por cada producto cuya suma del stock disponible y del reservado sea menor o igual al punto de reposición. La cantidad a reponer será la que permita llegar al stock máximo del producto y el estado de la solicitud se grabará como pendiente. La fecha de solicitud corresponderá a la fecha actual. El procedimiento deberá retornar `false` en caso de que ya exista previamente alguna solicitud de reposición en estado pendiente para la fecha actual, y `true` en caso contrario.
- **envio de emails a clientes:** el trabajo práctico deberá proveer la funcionalidad de generar emails para ser enviados a la dirección de email de le cliente—en la tabla `envio_emails`—cuando sucedan las siguientes novedades:
  - Cada vez que se haga efectivo el cierre de un pedido, se debe ingresar automáticamente un email con el asunto '`Pedido aceptado`', e indicando en el cuerpo del email los datos del pedido y el detalle de los productos solicitados.
  - Cada vez que un pedido sea cancelado, se debe ingresar automáticamente un email con el asunto '`Pedido cancelado`', e indicando en el cuerpo del email los datos del pedido y el detalle de productos.

- Cada vez que un pedido sea entregado, se debe ingresar automáticamente un email con el asunto '`Pedido entregado`', e indicando en el cuerpo del email los datos del pedido y el detalle de productos.

- Una vez al día, se debe generar un email para les clientes que hayan recibido satisfactoriamente su pedido el día anterior. El asunto debe ser '`Encuesta de satisfacción`', incluyendo en el cuerpo del email un texto breve que le invite a completar una encuesta, en alguna dirección URL.

Se deberá crear una tabla con datos de entrada de las transacciones para probar el sistema, que deberá contener los siguientes atributos: `id_orden`, `operacion`, `id_usuarie`, `id_direccion_entrega`, `id_pedido`, `id_producto`, `cantidad`, `fecha_hora_entrega`. A partir de esta tabla, se deberá hacer un procedimiento de testeo que invoque a las transacciones correspondientes de acuerdo a la operación. Los datos deberán tomarse del archivo `trx_pedidos` provisto en el directorio compartido de Google Drive.

Todo el código SQL escrito para este trabajo práctico, deberá poder ejecutarse desde una aplicación CLI escrita en Go.

5 JSON y Bases de datos NoSQL

Por último, para poder comparar el modelo relacional con un modelo no relacional NoSQL, se pide guardar los datos de clientes, direcciones de entrega, productos y pedidos (al menos dos pedidos, mínimo de tres productos por pedido) en una base de datos NoSQL basada en JSON. Para ello, utilizar la base de datos BoltDB. Este código también deberá ejecutarse desde una aplicación CLI escrita en Go.

6 Condiciones de Entrega

- El trabajo es grupal, en grupos de, *exactamente*, 4 integrantes. Se debe realizar en un repositorio privado `git`, hosteado en **Gitlab** con el apellido de les cuatro integrantes, separados con guiones, seguidos del string '-db1' como nombre del proyecto, e.g. `giunta-maradona-palermo-riquelme-db1`. Agregar a los docentes de la materia, los usernames de Gitlab `hdr` y `danbert` en el repo como `owner`'s.
- La fecha de entrega *máxima* es el 16 de noviembre de 2023 a las 1300 hs, con una defensa presencial del trabajo práctico por cada grupo, en la cual los docentes de la materia van a mirar lo que se encuentre en el repo git hasta ese momento, y hacer distintas preguntas.
- El informe del trabajo práctico se debe presentar en formato Asciidoc. Para ello, cuentan con una guía en **hdr.gitlab.io/adoc**.
- *Investigación:* En este trabajo práctico van a tener que investigar por su cuenta cómo se hacen algunas cosas en PostgreSQL. **No busquen en Stack Overflow ó sitios similares, para eso tienen la documentación oficial de PostgreSQL.**
- Uso apropiado de git: **no usar branches, ni rebase**. Asegurarse de configurar merge, nombre y apellido, y email:

```
$ git config --global pull.rebase false
$ git config --global user.email 'cristina@kirchner.com'
$ git config --global user.name 'Cristina Kirchner'
```